

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №2
з дисципліни
«Об'єктно-орієнтоване програмування»
на тему
«Розробка графічного редактора об'єктів»

Виконав:

студент групи ІМ-52
Пращук Данило Павлович
Номер в списку - 16

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання:

1. Створити у середовищі розробки новий проект з ім'ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.

Завдання згідно варіанту:

$N = 16 + 100 = 116$.

$16 \bmod 3$ не дорівнює 0, масив статичний.

"Гумовий" слід при вводі об'єктів - суцільна лінія чорного кольору для варіантів ($16 \bmod 4 = 0$).

Увід прямокутника - по двом протилежним кутам для варіантів ($16 \bmod 2 = 0$).

Відображення прямокутника - чорний контур з кольоровим заповненням для ($16 \bmod 5 = 1$).

Кольори заповнення прямокутника - помаранчевий для ($16 \bmod 6 = 4$).

Увід еліпсу - від центру до одного з кутів охоплюючого прямокутника для варіантів ($16 \bmod 2 = 0$).

Відображення еліпсу - чорний контур з білим заповненням для ($16 \bmod 5 = 1$).

Позначка поточного типу об'єкту, що вводиться - в меню (метод `OnInitMenuPopup`) для варіантів ($16 \bmod 2 = 0$).

main.cpp

```
#include <QApplication>
#include "mainWindow.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    MainWindow window;

    window.show();
    return app.exec();
}
```

mainWindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QActionGroup>
#include "shape.h"

class QPaintEvent;
class QMouseEvent;

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

protected:
    void paintEvent(QPaintEvent *event) override;
    void mousePressEvent(QMouseEvent *event) override;
    void mouseMoveEvent(QMouseEvent *event) override;
    void mouseReleaseEvent(QMouseEvent *event) override;
private:
    bool isDragging = false;
    QPoint startPt;
    QPoint endPt;
    QActionGroup *shapeGroup = nullptr;
}
```

```

static const int N = 116;
Shape* currentShape = nullptr;
Shape *pcshape[N] = {nullptr};
int shapeCount = 0;
};

#endif

```

mainWindow.cpp

```

#include "mainWindow.h"
#include <QMenu>
#include <QMenuBar>
#include <QMessageBox>
#include <QMouseEvent>
#include <QPaintEvent>
#include <QPainter>
#include <QRect>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent)
{
    setWindowTitle("Laba 2");
    resize(640, 480);

    QMenuBar *menu = menuBar();
    QMenu *menuFile = menu->addMenu("&Файл");
    QAction *fileAction = menuFile->addAction("про файл");
    QObject::connect(fileAction, &QAction::triggered, [this]()
        { QMessageBox::information(this, "File", "Це файл другої лабораторної з ооп, де
        потрібно створити базовий графічний редактор"); });

    QMenu *menuObjects = menu->addMenu("&Об'єкти");
    QAction *action1 = menuObjects->addAction("Крапка");
    QAction *action2 = menuObjects->addAction("Лінія");
    QAction *action3 = menuObjects->addAction("Прямокутник");
    QAction *action4 = menuObjects->addAction("Еліпс");
    QMenu *menuInfo = menu->addMenu("&Довідка");
    QAction *infoAction = menuInfo->addAction("довідкова інформація");
    QObject::connect(infoAction, &QAction::triggered, this, [this]() {
        QMessageBox::information(this, "Довідка", "Лабораторна робота №2 «Поліморфізм та
        об'єктна ієрархія фігур у графічному редакторі».<br>Базовий графічний
        редактор<br>Варіант: 16<br>Автор: Пращук Данило Павлович ІМ-52");
    });
    shapeGroup = new QActionGroup(this);

```

```

shapeGroup->setExclusive(true);

for (QAction *act : {action1, action2, action3, action4})
{
    act->setCheckable(true);
    shapeGroup->addAction(act);
}

connect(action1, &QAction::triggered, this, [this]()
        { delete currentShape;
          currentShape = new PointShape(); });
connect(action2, &QAction::triggered, this, [this]()
        { delete currentShape;
          currentShape = new LineShape(); });
connect(action3, &QAction::triggered, this, [this]()
        {
            delete currentShape;
            currentShape = new RectShape(); });
connect(action4, &QAction::triggered, this, [this]()
        {
            delete currentShape;
            currentShape = new EllipseShape(); });
}

MainWindow::~MainWindow()
{
    delete currentShape;
    for (int i = 0; i < shapeCount; i++)
    {
        delete pcshape[i];
    }
}

void MainWindow::mousePressEvent(QMouseEvent *event)
{
    if (event->pos().y() < menuBar()->height())
        return;

    if (event->button() == Qt::LeftButton && currentShape != nullptr)
    {
        if (shapeCount >= N)
            return;

        startPt = event->pos();
        endPt = event->pos();
        isDragging = true;

        if (dynamic_cast<PointShape*>(currentShape))

```

```

    {
        pcshape[shapeCount] = currentShape->clone();
        pcshape[shapeCount]->set(event->pos().x(), event->pos().y(), event->pos().x(), event->pos().y());
        shapeCount++;
        isDragging = false;
        update();
    }
}
}

```

```

void MainWindow::mouseMoveEvent(QMouseEvent *event)
{
    if (isDragging)
    {
        endPt = event->pos();
        update();
    }
}

```

```

void MainWindow::mouseReleaseEvent(QMouseEvent *event)
{
    if (event->button() == Qt::LeftButton && isDragging)
    {
        isDragging = false;
        endPt = event->pos();

        if (currentShape && shapeCount < N) {
            pcshape[shapeCount] = currentShape->clone();
            pcshape[shapeCount]->set(startPt.x(), startPt.y(), endPt.x(), endPt.y());
            shapeCount++;
            update();
        }
    }
}

```

```

void MainWindow::paintEvent(QPaintEvent *event)
{
    QMainWindow::paintEvent(event);
    QPainter painter(this);
    int menuHeght = menuBar()->height();

    QRect drawingArea(0, menuHeght, width(), height() - menuHeght);
    painter.setClipRect(drawingArea);

    for (int i = 0; i < shapeCount; i++)
    {
        if (pcshape[i])

```

```

    {
        pcshape[i]->show(painter);
    }
}

if (isDragging && currentShape)
{
    currentShape->set(startPt.x(), startPt.y(), endPt.x(), endPt.y());
    currentShape->rubber(painter);
}
}

```

shape . h

```

#ifndef SHAPE_H
#define SHAPE_H

#include <QPainter>
#include <QRect>
#include <QPoint>

class Shape
{
protected:
    long xs1, ys1, xs2, ys2;

public:
    Shape();
    virtual ~Shape() = default;

    void set(long x1, long y1, long x2, long y2);
    virtual void show(QPainter &painter) = 0;
    virtual void rubber(QPainter &painter);
    virtual Shape* clone() const = 0;
};

class PointShape : public Shape
{
public:
    void show(QPainter &painter) override;
    Shape* clone() const override;
};

class LineShape : public Shape
{
public:
    void show(QPainter &painter) override;

```

```
    void rubber(QPainter &painter) override;
    Shape* clone() const override;
};
```

```
class RectShape : public Shape
{
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
    Shape* clone() const override;
};
```

```
class EllipseShape : public Shape
{
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
    Shape* clone() const override;
};
```

```
#endif
```

shape . cpp

```
#include "shape.h"
#include <cmath>
#include <QPen>
#include <QColor>
```

```
Shape::Shape() : xs1(0), ys1(0), xs2(0), ys2(0) {}
```

```
void Shape::set(long x1, long y1, long x2, long y2)
{
    xs1 = x1;
    ys1 = y1;
    xs2 = x2;
    ys2 = y2;
}
```

```
void Shape::rubber(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 1, Qt::SolidLine));
    painter.setBrush(Qt::NoBrush);
}
```

```

void PointShape::show(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 6, Qt::SolidLine, Qt::RoundCap));
    painter.drawPoint(xs1, ys1);
}
Shape *PointShape::clone() const { return new PointShape(*this); }

```

```

void LineShape::show(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 2, Qt::SolidLine));
    painter.drawLine(xs1, ys1, xs2, ys2);
}
void LineShape::rubber(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 1, Qt::SolidLine));
    painter.drawLine(xs1, ys1, xs2, ys2);
}
Shape *LineShape::clone() const { return new LineShape(*this); }

```

```

void RectShape::show(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 2, Qt::SolidLine));
    painter.setBrush(QColor(255, 165, 0));
    QRect rect(QPoint(xs1, ys1), QPoint(xs2, ys2));
    painter.drawRect(rect.normalized());
}
void RectShape::rubber(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 1, Qt::SolidLine));
    painter.setBrush(Qt::NoBrush);
    QRect previewRect(QPoint(xs1, ys1), QPoint(xs2, ys2));
    painter.drawRect(previewRect.normalized());
}
Shape *RectShape::clone() const { return new RectShape(*this); }

```

```

void EllipseShape::show(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 2, Qt::SolidLine));
    painter.setBrush(Qt::white);
    int rx = std::abs(xs2 - xs1);
    int ry = std::abs(ys2 - ys1);
    painter.drawEllipse(QPoint(xs1, ys1), rx, ry);
}
void EllipseShape::rubber(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 1, Qt::SolidLine));
}

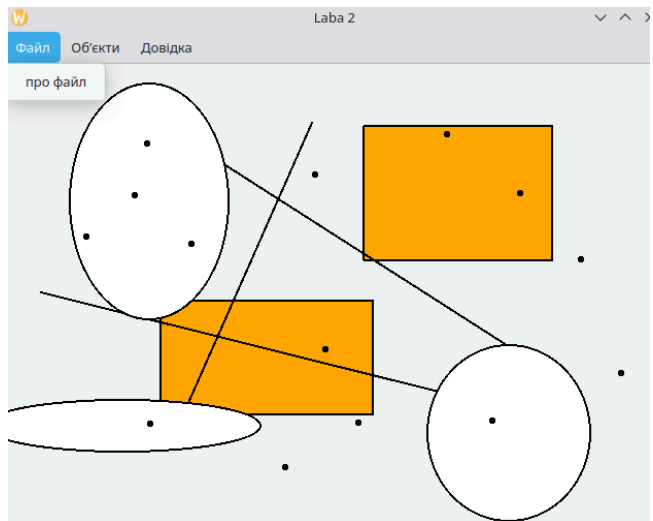
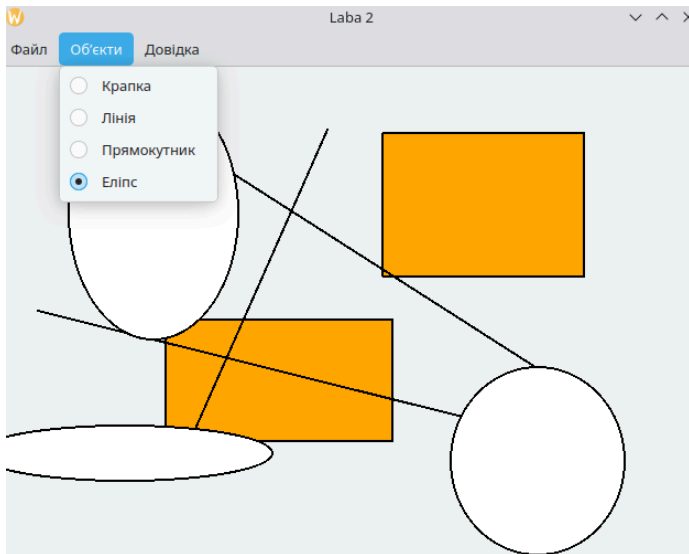
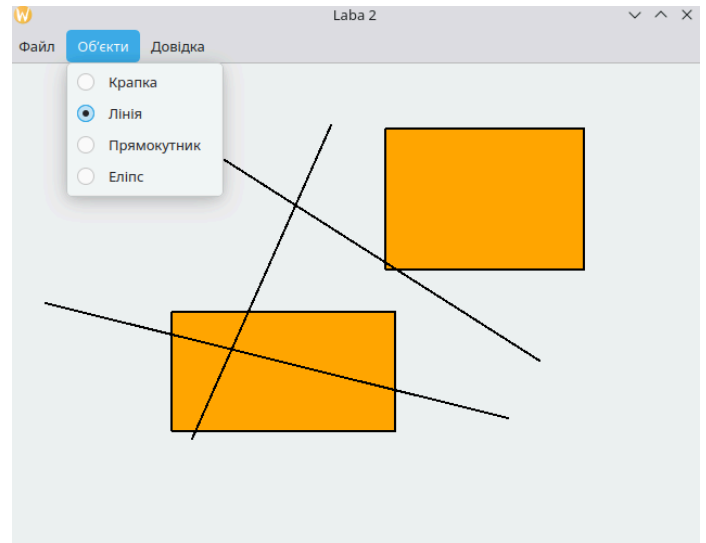
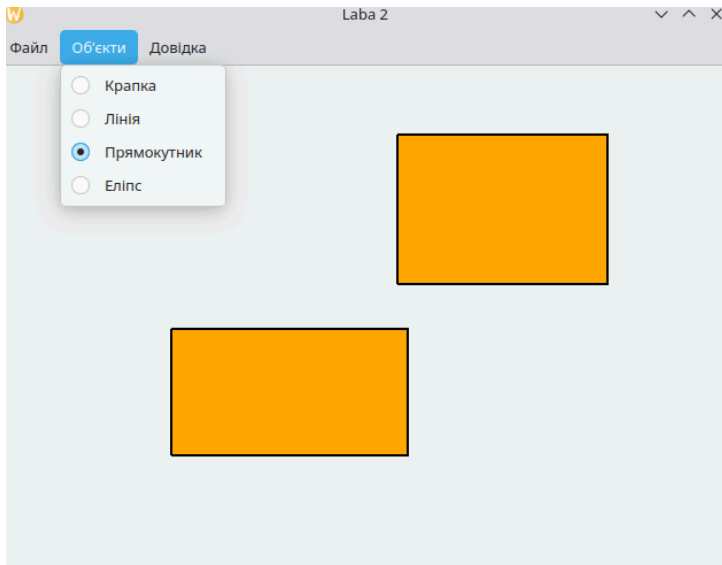
```

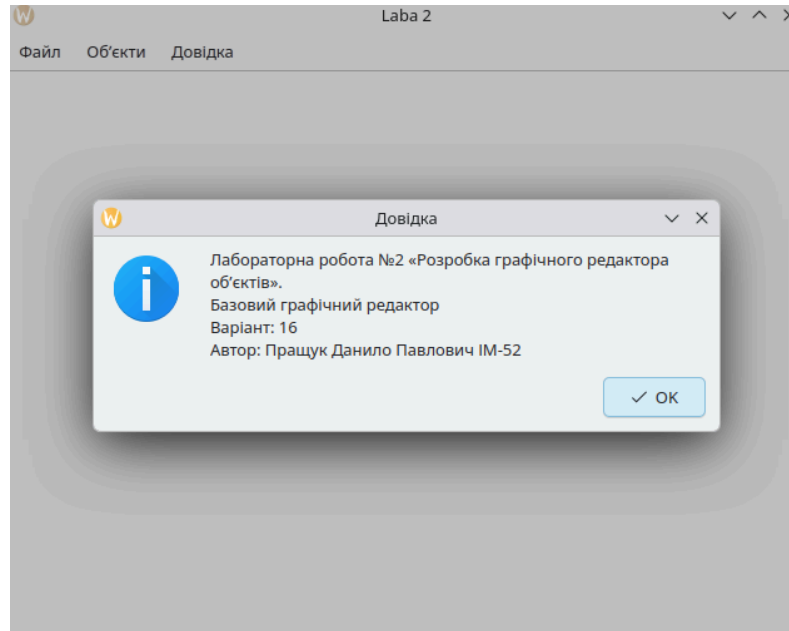
```

painter.setBrush(Qt::NoBrush);
int rx = std::abs(xs2 - xs1);
int ry = std::abs(ys2 - ys1);
painter.drawEllipse(QPoint(xs1, ys1), rx, ry);
}
Shape *EllipseShape::clone() const { return new EllipseShape(*this); }

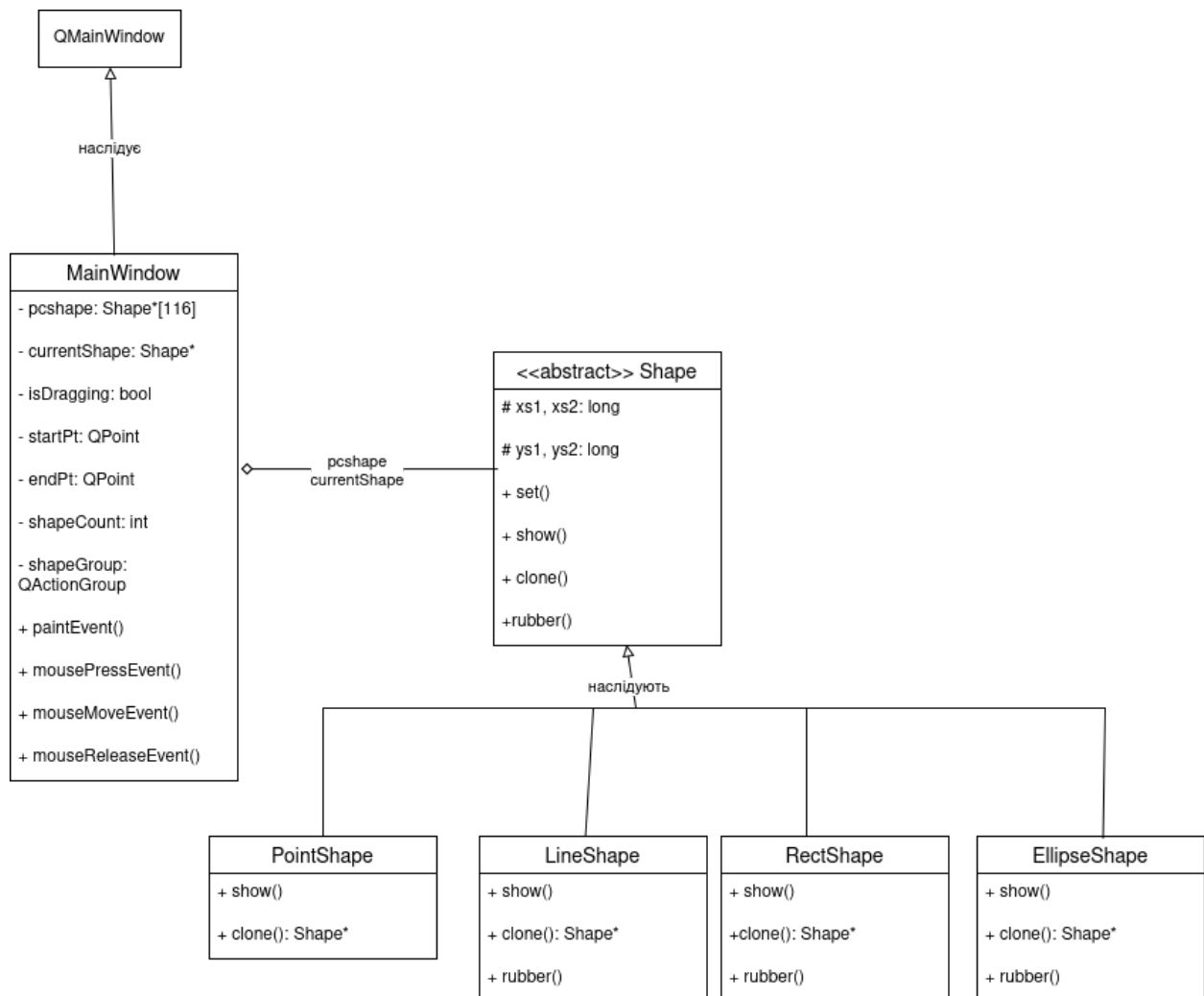
```

Скріншоти





Діаграма залежностей класів



Висновки:

Під час виконання лабораторної роботи було створено графічний редактор на базі Qt та C++, у якому за допомогою поліморфізму, віртуальних методів (show, rubber). Усі вимоги варіанта 16, зокрема ексклюзивна позначка інструмента в меню, суцільний гумовий слід, специфічні кольори заповнення та безпечне очищення пам'яті через віртуальний деструктор, виконано в повному обсязі.